

Acelerador em FPGA para o Supervisor de Segurança

LiDAR de uma Cadeira de Rodas Semi-Assistida: Co-Projeto Hardware–Software sobre Zynq-7000

Vítor Gonçalves de Andrade Silva
Engenharia Eletrônica — FCTE/UnB
Email: 261025182@aluno.unb.br

Thiago Henrique Oliveira Souza
Engenharia Eletrônica — FCTE/UnB
Email: 180131672@aluno.unb.br

Resumo—Este trabalho acelera em FPGA o núcleo do supervisor de segurança de uma cadeira de rodas motorizada semi-assistida. O supervisor decide, a cada varredura de LiDAR, se o avanço comandado pelo usuário deve ser reduzido ou interrompido: é o bloco de maior custo do sistema e o único cuja latência tem consequência física direta, pois determina a distância percorrida antes da frenagem. O algoritmo projeta os 720 feixes da varredura no referencial do veículo e testa, para 19 direções candidatas de desvio, se cada ponto cai no corredor que a cadeira ocupará — 9918 avaliações por varredura. Propõe-se um particionamento em que o custo $O(N \cdot K)$ migra para hardware e o custo $O(K)$ (função custo e escolha do candidato) permanece no ARM, onde residem os ganhos ajustáveis em campo. A arquitetura emprega um CORDIC pipelinado de 16 estágios, que não consome multiplicadores, alimentando 19 pistas paralelas cujas rotações têm coeficientes constantes de síntese. A comunicação usa AXI4-Stream para os feixes (via AXI-DMA) e AXI4-Lite para calibração e resultados. A equivalência numérica foi verificada bit a bit contra um modelo de referência em ponto fixo, sobre três varreduras *reais* capturadas na cadeira; as três passam. A análise de precisão revelou um defeito real de formato: o ângulo do feixe atinge $-5,85$ rad e não cabe em Q3.13, saturando e projetando o ponto no lugar errado (erro de 402 mm); com o ângulo envolvido em $[-\pi, \pi)$ o erro cai a 0,99 mm médio, cerca de um LSB. Medido na Raspberry Pi 5, o software gasta 3,54 ms (escalar) ou 0,488 ms (vetorizado); o hardware conclui em 751 ciclos (7,51 μ s a 100 MHz), $472\times$ e $65\times$ mais rápido, respectivamente.

I. INTRODUÇÃO

Cadeiras de rodas motorizadas são inacessíveis a uma parcela expressiva dos usuários que mais precisariam delas: pesquisa clínica clássica aponta que profissionais de reabilitação consideram muitos pacientes inaptos ao uso justamente pela inadequação das interfaces de controle [1]. Espasmos e comandos acidentais no joystick produzem acelerações repentinas e colisões.

O projeto no qual este trabalho se insere converte uma cadeira comercial em um veículo semi-assistido sob um princípio de segurança explícito: o sistema de alto nível pode **frear** ou **desviar**, mas **nunca acelerar**; a iniciativa de movimento é sempre do usuário. Um supervisor executando em Raspberry Pi 5 com ROS 2 [6] lê um LiDAR RPLIDAR C1 e

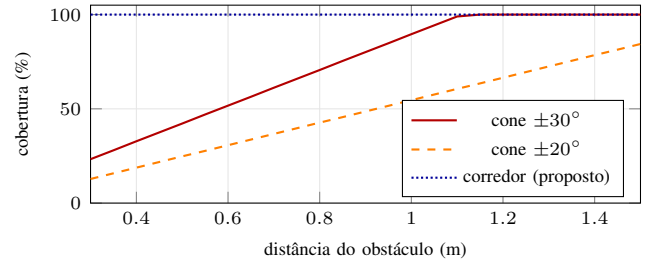


Figura 1. Fração da largura da cadeira efetivamente monitorada. O cone ancorado no sensor *piora* quanto mais perto está o obstáculo, porque estreita enquanto o desvio lateral do vértice permanece constante: a 0,60 m cobre 52 %, e a parcela não coberta é sempre a metade direita. O teste de corredor cobre 100 % por construção.

atenua o comando do joystick antes de repassá-lo ao firmware de tempo real em ESP32-S3.

A. O bloco a acelerar e por quê

O supervisor é, disparadamente, o nó mais pesado do sistema: medimos 17,7 % da CPU da Pi 5 na implementação escalar e 8,3 % após vetorização. Mais importante que o custo, porém, é a natureza da sua latência: *ela tem consequência física*. O supervisor só corta o comando de avanço; a cadeira ainda percorre uma distância por inércia. Todo atraso entre a chegada da varredura e a decisão se traduz em centímetros adicionais rumo ao obstáculo. Trata-se, portanto, de um caso em que acelerar não é otimização, e sim segurança — o que motiva o co-projeto.

A escolha é ainda mais pertinente porque uma revisão recente do supervisor revelou uma falha geométrica de segurança. A folga frontal era medida por um *cone angular* ancorado no sensor, construção válida apenas se o LiDAR estivesse sobre o eixo longitudinal do veículo. Estando ele 0,336 m fora do eixo, o cone varre uma faixa deslocada e a metade direita da cadeira fica cega justamente nas curtas distâncias (Fig. 1). A correção substituiu o cone pelo teste do *corredor físico* que a cadeira ocupará — e é esse algoritmo, correto porém mais caro, que se acelera aqui.

Tabela I
PARTICIONAMENTO HW/SW

Etapa	Custo	Onde
Projeção + teste de corredor	$O(N \cdot K) = 9918$	FPGA
Função custo + $\argmin$	$O(K) = 19$	ARM

B. Trabalhos relacionados

A função custo que pontua direções candidatas pela folga é aparentada ao *Vector Field Histogram* [2], com a distinção essencial de raciocinar sobre a *planta* do robô e não sobre um cone do sensor. O CORDIC [3], [4] é a escolha clássica para trigonometria em FPGA sem multiplicadores. O uso de aceleradores acoplados a processadores embarcados via AXI4 sobre Zynq segue a prática consolidada em [5].

II. DESENVOLVIMENTO

A. O algoritmo

Cada retorno do LiDAR, um par (r_i, a_i) , é projetado no referencial `base_link` (origem no centro do eixo traseiro):

$$X_i = x_L + r_i \cos(a_i + \theta_L), \quad Y_i = y_L + r_i \sin(a_i + \theta_L), \quad (1)$$

onde (x_L, y_L, θ_L) é a pose calibrada do sensor. Para avaliar um candidato de desvio δ_k , giram-se os pontos por $-\delta_k$ em torno do eixo traseiro — o centro instantâneo de rotação do acionamento diferencial:

$$x_i^{(k)} = X_i \cos \delta_k + Y_i \sin \delta_k, \quad (2)$$

$$y_i^{(k)} = -X_i \sin \delta_k + Y_i \cos \delta_k. \quad (3)$$

O ponto é obstáculo se $|y_i^{(k)}| \leq W/2$ e $x_i^{(k)} > X_f$, com folga $x_i^{(k)} - X_f$; a folga do candidato é o mínimo sobre os pontos. Aqui $W = 0,61$ m é a largura da cadeira e $X_f = 0,667$ m a frente do chassi. Note que a estrutura da própria cadeira cai atrás de X_f e é descartada por construção, sem nenhum limiar arbitrário de distância mínima.

Com $N = 720$ feixes e $K = 19$ candidatos (δ_k de -45° a $+45^\circ$, passo 5°), são 9918 avaliações por varredura — o custo $O(N \cdot K)$ que motiva o acelerador.

B. Particionamento Hardware–Software

O supervisor decompõe-se em duas partes de custo muito distinto (Tab. I). Apenas a primeira migra para hardware.

A decisão de *não* levar tudo para hardware é deliberada. A função custo são 19 comparações: o ganho seria irrisório. Em compensação, é nela que residem os pesos ajustados em campo (w_{obs} , w_{dev} , K_{assist}), retocados a cada ensaio. Congelá-los em RTL trocaria flexibilidade por nada — exatamente o *trade-off* que o co-projeto existe para arbitrar.

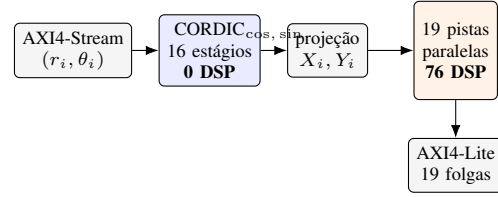


Figura 2. *Datapath*. Vazão de **1 feixe/ciclo**, sem bolhas. O CORDIC é compartilhado (custo $O(N)$); as pistas são replicadas (custo $O(N \cdot K)$).

Tabela II
FORMATOS ADOTADOS

Grandeza	Formato	Faixa	LSB
distâncias, coords.	Q6.10	± 32 m	0,98 mm
cos, sin	Q1.14	± 2	$6,1 \times 10^{-5}$
ângulos	Q3.13	$\pm 4,0$ rad	$1,22 \times 10^{-4}$ rad

C. Arquitetura de hardware

A Fig. 2 mostra o *datapath*. O CORDIC é **compartilhado** porque a projeção (1) é comum a todos os candidatos: o custo $O(N)$ é pago uma única vez. O que se replica são as 19 pistas, que carregam o custo $O(N \cdot K)$ — e é precisamente aí que o paralelismo espacial da FPGA rende: as pistas avaliam *no mesmo ciclo* aquilo que o processador precisa iterar.

a) **CORDIC**: Optou-se por CORDIC em modo rotação [3], pipelinado em 16 estágios, em vez de uma LUT de seno. Uma LUT exigiria uma ROM indexada pelo índice do feixe, o que só funciona se `angle_min`, o incremento e o `yaw` forem fixos em síntese — mas eles são **parâmetros de calibração**, reescritos em tempo de execução por AXI4-Lite. O CORDIC aceita qualquer ângulo e, por usar apenas somadores e deslocamentos cabeados, não consome *nenhum* DSP48. Como o laço só converge para $|z| \leq 1,7434$ rad, um estágio de redução de quadrante traz θ para $[-\pi/2, \pi/2]$ e inverte o sinal na saída.

b) **Pistas**: Em cada pista, $\cos \delta_k$ e $\sin \delta_k$ são **constantes de síntese** (*generics*): a rotação (3) vira quatro multiplicações de coeficiente fixo, mapeadas diretamente em DSP48. Daí $19 \times 4 = 76$ DSP, mais 2 da projeção: 78 de 220 disponíveis no `xc7z020` (35 %).

D. Formato de ponto fixo

Os formatos adotados estão na Tab. II. A escolha da *faixa* de Q3.13 mostrar-se-á decisiva na Seção III-B.

E. Interfaces AXI4

Os feixes chegam por **AXI4-Stream** (`TDATA = [\theta(31:16) | r(15:0)]`, `TUSER` = feixe válido, `TLAST` = fim da varredura), alimentado por um AXI-DMA que lê a varredura da DDR. O controle e o resultado usam **AXI4-Lite**: `CTRL` (*start*), `STATUS` (*done/busy*) e 19 registradores de folga, com sinal estendido a 32 bits. Uma interrupção sinaliza a conclusão, dispensando espera ocupada no ARM. A escolha é natural: *stream* para o fluxo volumoso e sequencial (a varredura),

Tabela III
ERRO DO PONTO FIXO CONTRA A REFERÊNCIA EM PONTO FLUTUANTE

	erro médio	erro máximo
θ saturado (defeito)	17,89 mm	402,15 mm
θ envolvido (corrigido)	0,99 mm	5,27 mm

mapeado em memória para o punhado de escalares (calibração e resultado).

F. Sistema em Chip

O acelerador foi empacotado como IP e integrado ao ARM Cortex-A9 do Zynq num *block design* (Fig. 3). O ARM mantém a varredura na DDR; o AXI-DMA a lê pela porta S_AXI_HP0 e a entrega ao acelerador como *stream*; as 19 folgas retornam por AXI4-Lite e a IRQ_F2P sinaliza a conclusão.

III. RESULTADOS

A. Protocolo de verificação

A verificação é feita por *equivalência bit a bit* contra um modelo de referência, e não por inspeção de formas de onda. O fluxo é:

- 1) Um **modelo em ponto fixo** (Python) emula o RTL exatamente: mesmo CORDIC, mesmos deslocamentos, mesmos truncamentos.
- 2) Um **modelo em ponto flutuante** serve de referência física, para medir o erro de quantização.
- 3) O modelo gera o **estímulo** e o **esperado**; o testbench VHDL lê os arquivos, injeta os 720 feixes e compara as 19 folgas.
- 4) As constantes do RTL (`cordic_pkg.vhd`) são **geradas** pelo mesmo modelo, de modo que RTL e referência casam *por construção*, e não por transcrição manual — um erro de digitação numa tabela de constantes é, aqui, indetectável por inspeção.

O estímulo não é sintético: são três varreduras **reais** da cadeira, capturadas com um balde posicionado 21 cm à direita do eixo — exatamente o ponto cego do supervisor antigo (Fig. 4). As três passam com **zero divergência**.

B. Precisão: um defeito de formato

A análise de precisão produziu o achado mais instrutivo do trabalho. O primeiro modelo em ponto fixo divergia da referência em até 402 mm — inaceitável num sistema de segurança, e grande demais para ser arredondamento.

A causa não era ruído de quantização, e sim **estouro de faixa**. O ângulo do feixe no referencial do sensor, $\theta_i = a_{\min} + i \cdot \Delta a + \theta_L$, atinge $-5,85$ rad; o formato Q3.13 comporta apenas $\pm 4,0$ rad. O valor **saturava**, e o feixe era projetado numa direção completamente errada. Envolvendo θ em $[-\pi, \pi)$ antes da quantização — operação barata, feita no host — o erro desaba (Tab. III).

O erro final é da ordem de **um LSB** de Q6.10 (0,98 mm) e representa 0,9 % da zona de parada de 600 mm — ou seja, a

Tabela IV
TEMPO POR VARREDURA E GANHO

Implementação	Tempo	Speedup	
		vs. escalar	vs. NumPy
Software escalar (Pi 5)	3,54 ms	—	—
Software vetorizado (NumPy)	0,488 ms	7,3×	—
HW 100 MHz (751 ciclos)	7,51 μs	472×	65×
HW 150 MHz	5,01 μ s	707×	98×

quantização *não* é o fator limitante do sistema. A lição é que, em ponto fixo, a escolha da **faixa** costuma ser mais crítica que a da **resolução**: perdemos 402 milímetros por três bits de parte inteira, não por falta de bits fracionários.

A Fig. 5 confronta, candidato a candidato, a folga calculada em software (ponto flutuante, a referência física) com a produzida pelo **hardware** — e o “hardware” aqui é a saída efetiva da simulação VHDL, lida de `rtl_out`, e não o modelo. As curvas se sobrepõem: é esse o resultado. A Fig. 6 mostra o resíduo, com sinal, para as três varreduras: 57 comparações, erro médio de 0,99 mm e máximo de 5,27 mm, todas dentro de poucos LSB.

C. Desempenho

O *baseline* de software foi medido na própria Raspberry Pi 5 que embarca o sistema; a latência de hardware é o número de ciclos **medido na simulação** (7515 ns a 100 MHz), não uma estimativa analítica. A Tab. IV consolida os tempos.

Vale interpretar o ganho, e não apenas exibi-lo (Fig. 7). O LiDAR entrega uma varredura a cada 100 ms: mesmo o software vetorizado consome apenas 0,5 % desse orçamento, de modo que o **acelerador não “salva” o sistema** — ele não estava perdendo prazos. O valor real é outro, e triplo: (i) libera 8,3 % da CPU num processador que também roda SLAM e EKF, e que já apresentou congelamento por exaustão de memória; (ii) torna a latência **determinística**, ao contrário de um nó Python sujeito ao escalonador de um Linux não-preemptivo — e determinismo, num laço de segurança, vale mais que média baixa; (iii) abre orçamento para aumentar K ou a densidade angular sem custo de tempo, já que o gargalo passa a ser a taxa do sensor.

D. Síntese: recursos, timing e potência

Alvo: `xc7z020clg484-1` (ZedBoard), período de 10 ns. Os valores a seguir vêm dos relatórios do Vivado (`vivado/reports/`).

A síntese conclui sem erros nem *critical warnings*, e o roteamento fecha com **WNS de 0,872 ns** — folga confortável para o alvo de 100 MHz ($F_{\max} = 109,6$ MHz). O consumo de lógica é modesto: 4,6 % dos LUT e 1,7 % dos *flip-flops*.

a) **Potência**: O acelerador consome 0,246 W no total, dos quais apenas 0,140 W são dinâmicos — o restante é a corrente de fuga do dispositivo, que existiria com a FPGA ligada mesmo sem nenhum projeto carregado. A parcela dinâmica é baixa porque o *datapath*, embora largo (19 pistas em paralelo), é raso: cada estágio faz uma multiplicação e uma

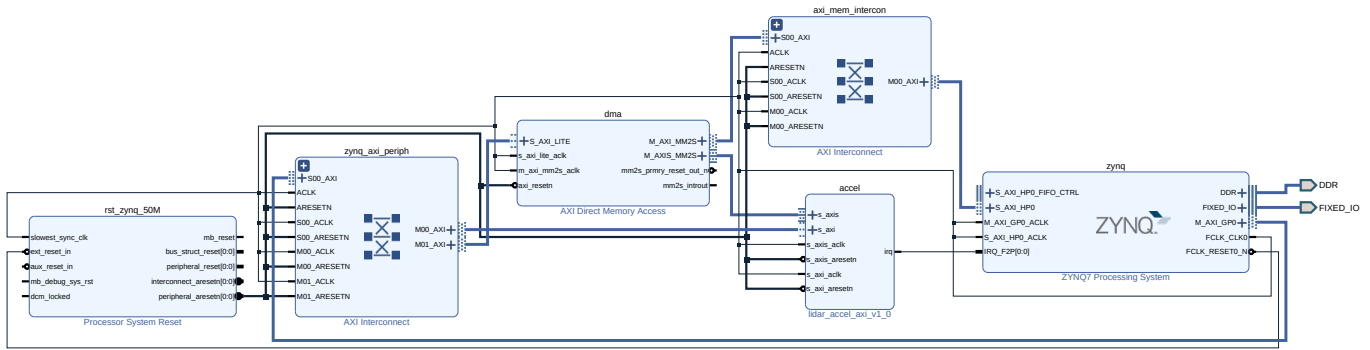


Figura 3. *Block design* no Vivado. O ARM (zynq) alcança o acelerador e o DMA por AXI4-Lite através do zynq_axi_periph. O dma lê a varredura da DDR (M_AXI_MM2S → axi_mem_intercon → S_AXI_HP0) e a entrega ao accel por AXI4-Stream (M_AXIS_MM2S → s_axis). A conclusão retorna por irq → IRQ_F2P, dispensando espera ocupada no processador.

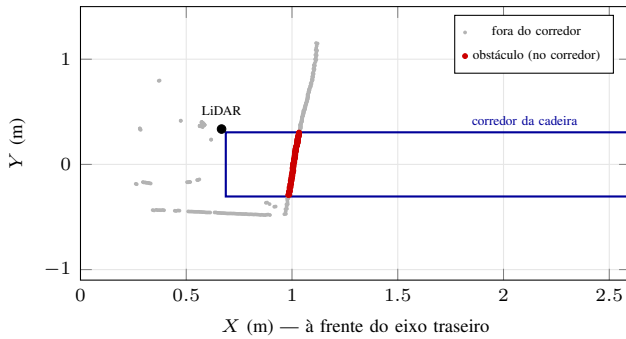


Figura 4. Varredura real projetada no base_link (estímulo do testbench). O LiDAR está 0,336 m à esquerda do eixo. Os 117 pontos em vermelho caem no corredor que a cadeira ocupará — entre eles o balde, que o cone antigo não enxergava.

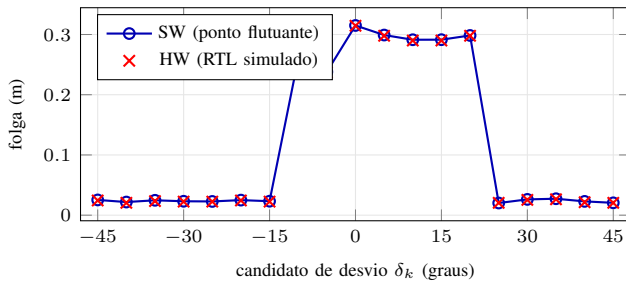


Figura 5. Folga por candidato: software *versus* saída real do RTL, para a varredura de teste. As curvas se sobrepõem em toda a faixa — o desvio é invisível nesta escala, o que é precisamente o resultado desejado.

comparação, sem memória e sem realimentação. Convém não superinterpretar o número: uma comparação honesta com a Raspberry Pi 5 exigiria medir a potência *incremental* da CPU ao executar o supervisor, o que não foi feito — e o Zynq também consumiria o seu próprio ARM. O que se pode afirmar é que o custo energético da lógica é desprezível frente aos motores da cadeira, da ordem de dezenas de W.

b) *Onde a previsão analítica errou — para mais:* Havia-se previsto 78 DSP48 (19 × 4 das pistas mais 2 da projeção). A ferramenta usou **54**, 31 % a menos. A razão é instrutiva

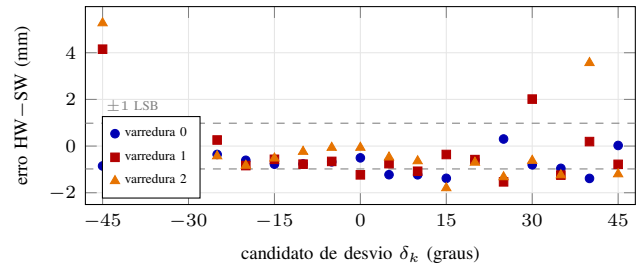


Figura 6. Resíduo HW-SW, com sinal, nas três varreduras (57 comparações). Linhas tracejadas: ± 1 LSB de Q6.10 (0,98 mm). O erro máximo, 5,27 mm, ocorre quando a quantização faz um ponto próximo da borda do corredor cruzar o limiar $|y| \leq W/2$, trocando qual ponto é o mínimo — efeito inerente a um critério de decisão por limiar, e não acúmulo de erro aritmético.

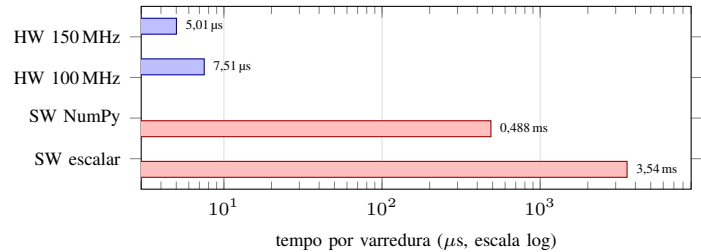


Figura 7. Tempo por varredura (escala logarítmica — são quase três ordens de grandeza). *Baseline* de software medido na própria Raspberry Pi 5; o tempo de hardware é o número de ciclos **medido na simulação**.

Tabela V
SÍNTESE E IMPLEMENTAÇÃO (VIVADO 2022.2, xc7z020c1g484-1)

Recurso	Previsto	Obtido	Disponível	%
LUT	—	2445	53 200	4,6 %
Flip-flop	—	1808	106 400	1,7 %
DSP48E1	78	54	220	24,5 %
BRAM	0	0	140	0 %
SRL	—	17	—	—

WNS (*setup*): **0,872 ns** ⇒ o *timing* fecha a 100 MHz
WHS (*hold*): 0,059 ns
F_{max}: **109,6 MHz**
Potência total: **0,246 W** (dinâmica 0,140 W + estática 0,106 W)

e só aparece ao ler o *DSP Final Report*: o DSP48E1 não é um multiplicador isolado, mas um bloco *multiplicador-acumulador* com uma porta *C* e uma cascata dedicada (PCIN) para o próximo DSP da coluna. A síntese reconheceu que

$$x^{(k)} = X \cos \delta_k + Y \sin \delta_k$$

é exatamente a forma que o bloco implementa nativamente, e fundiu o segundo produto e a soma em *um* DSP — ora pela porta *C* ($C+A*B$), ora encadeando pelo PCIN ($PCIN+A*B$). Onde o coeficiente é trivial, economizou ainda mais: a pista central ($\delta_9 = 0$, com $\sin \delta_9 = 0$) dispensa metade dos produtos.

A lição é que estimar recursos contando operadores do RTL *superestima* o custo: o mapeamento explora estrutura do dispositivo que não aparece no código. A folga resultante é substancial — sobram 75 % dos DSP, espaço para praticamente *triplicar* o número de candidatos *K* sem sair do xc7z020.

c) *Confirmações*: O CORDIC de fato não consome **nenhum** DSP — todos os 54 estão no `corridor_core` (pistas e projeção), como se pretendia ao escolher somadores e deslocamentos cabeados. E o uso de BRAM é **zero**, confirmando que a arquitetura *streaming* não precisa armazenar a varredura: cada feixe é consumido no ciclo em que chega.

IV. CONCLUSÃO

O acelerador atinge o objetivo: $472\times$ mais rápido que o software escalar e $65\times$ que o vetorizado, com equivalência **bit a bit** verificada contra um modelo de referência sobre dados reais da cadeira.

A análise crítica, porém, precisa ir além do número. **O ganho de tempo não era necessário**: o software já cabia folgadoamente no orçamento de 100 ms do sensor. O benefício legítimo é o alívio de CPU numa Pi que roda SLAM e EKF, e sobretudo o **determinismo** da latência num laço de segurança. Reportamos isso explicitamente porque a conclusão oposta — “aceleramos, logo o sistema melhorou” — seria uma leitura desonesta dos dados.

O resultado mais transferível não é o *speedup*, e sim o **defeito de faixa em Q3.13**: um ângulo de $-5,85$ rad saturando num formato de ± 4 rad produziu 402 mm de erro *silencioso* — o RTL não acusa, a simulação “roda”, e só a comparação com uma referência física revela. Foi o modelo de ouro, e não a inspeção de ondas, que o encontrou.

A. Trabalhos futuros

- 1) **Gravar em placa**: os resultados aqui são de simulação pós-síntese; falta o ensaio no ZedBoard, com o driver do ARM (`sw/lidar_accel.c`) sobre o AXI-DMA.
- 2) **Explorar o espaço de projeto**: com 35 % dos DSP usados, cabem *K* maior (mais candidatos) ou pistas com maior precisão; resta caracterizar o *trade-off* recursos $\times$ potência $\times$ qualidade da decisão.
- 3) **Reduzir a latência fim-a-fim**: o acelerador resolve $7,5\mu s$ de um laço cujo verdadeiro atraso é a 100 ms do sensor. O ganho maior estaria em processar a varredura *incrementalmente*, à medida que os feixes chegam —

algo que a arquitetura *streaming* já permite, e o software não.

REFERÊNCIAS

- [1] L. Fehr, W. E. Langbein e S. B. Skaar, “Adequacy of power wheelchair control interfaces for persons with severe disabilities: A clinical survey,” *J. Rehabil. Res. Dev.*, vol. 37, no. 3, pp. 353–360, 2000.
- [2] J. Borenstein e Y. Koren, “The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots,” *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, 1991.
- [3] J. E. Volder, “The CORDIC Trigonometric Computing Technique,” *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [4] R. Andracka, “A survey of CORDIC algorithms for FPGA based computers,” em *Proc. ACM/SIGDA Int. Symp. FPGAs*, 1998, pp. 191–200.
- [5] L. H. Crockett, R. A. Elliot, M. A. Enderwitz e R. W. Stewart, *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000*. Glasgow: Strathclyde Academic Media, 2014.
- [6] Open Robotics, “ROS 2 Jazzy Jalisco Documentation,” 2024. [Online]. Disponível: <https://docs.ros.org/en/jazzy/>